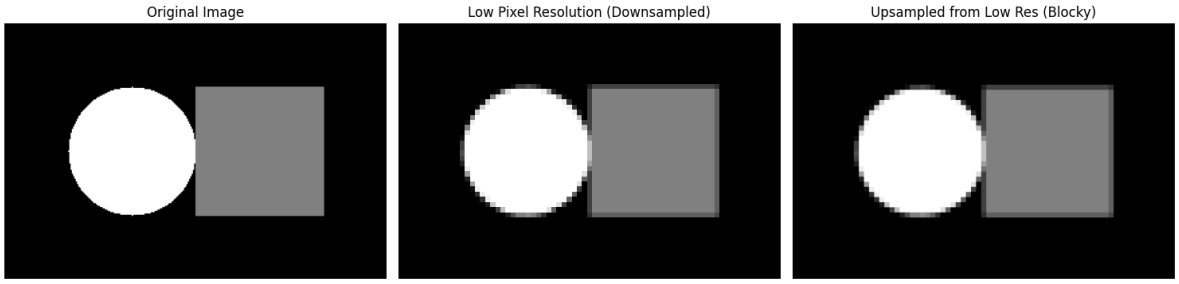
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Function to display images
def display_images(images, titles):
    plt.figure(figsize=(15, 5))
    for i, (img, title) in enumerate(zip(images, titles)):
        plt.subplot(1, len(images), i + 1)
        plt.imshow(img, cmap='gray')
        plt.title(title)
        plt.axis('off')
    plt.tight_layout()
    plt.show()

# --- 1. Pixel Resolution Demonstration ---

# Load a sample grayscale image (e.g., using a common image, or create a synth
# For demonstration, we'll try to load a known image. If it fails, we'll creat
try:
    # Download an image if not available locally
    import urllib.request
    url = "https://i.stack.imgur.com/r62bZ.png" # A sample grayscale image
    urllib.request.urlretrieve(url, "sample_medical_image.png")
    original_image = cv2.imread('sample_medical_image.png', cv2.IMREAD_GRAYSCA
    if original_image is None:
        raise FileNotFoundError
except (urllib.error.URLError, FileNotFoundError):
    print("Could not download sample image, generating a synthetic one.")
    # Generate a simple synthetic grayscale image
    original_image = np.zeros((200, 300), dtype=np.uint8)
    cv2.circle(original_image, (100, 100), 50, 255, -1)
    cv2.rectangle(original_image, (150, 50), (250, 150), 128, -1)


print(f"Original image dimensions: {original_image.shape}")

# Reduce pixel resolution (downsampling)
low_res_image = cv2.resize(original_image, (original_image.shape[1] // 4, orig
                           interpolation=cv2.INTER_AREA)
# Increase pixel resolution (upsampling) - typically used for display after do
high_res_display = cv2.resize(low_res_image, original_image.shape[::-1],
                              interpolation=cv2.INTER_NEAREST) # Nearest nei

display_images([original_image, low_res_image, high_res_display],
               ['Original Image', 'Low Pixel Resolution (Downsampled)', 'Upsar
```

```
Could not download sample image, generating a synthetic one.
```

```
Original image dimensions: (200, 300)
```




```
# --- 2. Intensity Resolution Demonstration ---

# Ensure the original image is 8-bit for demonstration, if it's not already.
# If it came from IMREAD_GRAYSCALE, it's likely 8-bit (0-255 levels).

# Reduce intensity resolution to 4-bit (16 gray levels)
# Max intensity for 8-bit is 255 (0-255)
# We want 16 levels, so each step is 256 / 16 = 16.
# Quantize by dividing by step, rounding, then multiplying by step.
levels_4bit = 16
intensity_4bit_image = np.floor(original_image / (256 / levels_4bit)) * (256
intensity_4bit_image = intensity_4bit_image.astype(np.uint8)

# Reduce intensity resolution to 2-bit (4 gray levels)
levels_2bit = 4
intensity_2bit_image = np.floor(original_image / (256 / levels_2bit)) * (256
intensity_2bit_image = intensity_2bit_image.astype(np.uint8)

display_images([original_image, intensity_4bit_image, intensity_2bit_image],
               ['Original (8-bit, 256 levels)', '4-bit (16 levels)', '2-bit
```